ICCS225

Database Fundamentals

Justin Copeland

Contents

1 Introduction **4**
Purposes of Database Systems . . . 4
View of Data . . . 4
Data Models . . . 4
Relational Data Model . . . 5
Data Abstraction . . . 5
Instances and Schemas . . . 6
Physical data independence . . . 6
Database Languages . . . 6
Data Definition Language . . . 6
Data Manipulation Language . . . 7
The SQL Language . . . 7
Database Access from Application Programs . . . 7
Database Design . . . 7
Database Engine . . . 7
The Storage Manager . . . 8
The Query Processor . . . 8
Transaction Management . . . 8
Database Architecture . . . 9
Database Administrator . . . 10

2 The Relational Model **11**
Structure of Relational Databases . . . 11
Database Schema . . . 11
Keys . . . 12
Relational Algebra . . . 13
The Select Operation . . . 13
The Project Operation . . . 13
Composition of Relational Operators . . . 14
The Cartesian Product Operation . . . 14
The Join Operation . . . 15
Set Operations . . . 16
The Assignment Operation . . . 17
The Rename Operation . . . 17
Equivalent Queries . . . 18

3 SQL **19**
Overview of SQL . . . 19
SQL Data Definition . . . 19

Basic Types . . . 20
Basic Schema Definition . . . 20
Basic Query Structure . . . 20
String Operations . . . 21
Ordering Display of Tuples . . . 22
Where Clause Predicates . . . 22
Set Operations . . . 22
Null Values . . . 23
Aggregate Functions . . . 23
Group By . . . 23
Having Clause . . . 24
Nested Subqueries . . . 24
Testing For Empty Relations . . . 25
Subqueries In the From Clause . . . 26
With Clause . . . 26
Scalar Subquery . . . 26
Deletion . . . 26
Insertion . . . 27
Updates . . . 27
Views . . . 28
Joining Relations . . . 29
Natural Join . . . 29
Inner/Outer Join . . . 30
Transactions . . . 31
Integrity Constraints . . . 31
Constraints on a Single Relation . . . 31
Check . . . 32
Referential Integrity . . . 32
Authorization . . . 32

4 Database Design Using the E-R Model 33
Design Phases . . . 33
Design Alternatives . . . 33
The Entity-Relationship Model . . . 34
Entity Sets . . . 34
Relationship Sets . . . 34
Complex Attributes . . . 35
Mapping Cardinalities . . . 36
Primary Key . . . 37
Primary Key for Entity Sets . . . 37
Primary Key for Relationship Sets . . . 38
Choice of Primary Key for Binary Relationships . . . 38
Weak Entity Sets . . . 38

5 Relational Database Design 40
Well-Structured Relations . . . 40
Normalization . . . 40
Steps in Normalization . . . 41
Functional Dependencies and Keys . . . 41
Candidate Keys . . . 41
Normalization Example . . . 42

Represent the View in Tabular Form 42
Convert to First Normal Form 42
Convert to Second Normal Form 43
Convert to Third Normal Form 44

1 | Introduction

Databases are everywhere. They are the backbone of systems all around the world. They are used in enterprise information, manufacturing, banking and finance, universities – practically anywhere you can think of. They are essential to keep track of taxes, employee salaries, student grades. Therefore, databases must be well designed to prevent damages.

Purposes of Database Systems

In the early days, database applications were built directly on top of file systems. This often led to things like

- *Data redundancy and inconsistency* – data stored in multiple file formats or styles
- *Difficulty in accessing data* – various new programs had to be written to carry out different tasks
- *Data isolation* – multiple files and formats
- *Integrity problems* – hard to add new constraints or change existing ones
- *Atomicity of updates* – failures may leave database in an inconsistent state with partial updates carried out
- *Concurrent access by multiple users* – uncontrolled concurrent accesses led to inconsistencies/data races
- *Security problems* – it was hard to provide a user access to some, but not all data

These difficulties led to the initial development of database systems back in the 1960s and 1970s.

View of Data

A database system is essentially a collection of interrelated data and a set of programs which allow users to access and modify said data. One major purpose of the database is to provide users with an abstract view of the data – one which leaves out all the nitty gritty details about how data is stored and maintained.

Data Models

Underlying the structure of the database is the *data model* – a collection of tools for describing data, data relationships, data semantics and data constraints. Some of these models include

- *Relational models* – which involve set theory / relational algebra

- *Entity-relationship data models* – which involve relationships between tables
- *Object-based data models* – which involve treating data as objects instead of tables

Relational Data Model

In the relational data model, all data are represented in the form of tables. Each table can have multiple columns, and each column has a unique name. Each row of the table represents a tuple of information. Take for example, the sample relational database below

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Table 1.1: The *instructor* table

dept_name	building	budget
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

Table 1.2: The *department* table

Data Abstraction

For the system to be usable, it must retrieve data efficiently. This desire of efficiency has led to many complex data structures being used to represent data in the database. Developers have hidden this complexity through several levels of *data abstraction*

- On the *physical level*, the lowest level of abstraction describes complex low-level data structures. Relations can be described as a block of consecutive bytes.
- The *logical level* describes what data are stored and what relationships exist among those data. The user at the logical level does not need to be aware of the implementation at the

physical level. At this level, relations and interrelations of said relations can be described by a type definition

- The *view level* is the highest level of abstraction, which describes only part of the entire database (since many users only need to view a small portion of it, this can simplify their interaction with the system). They can be thought of as fake, temporary tables in memory. At this level, users see a set of application programs that hide details of the data types.

Instances and Schemas

Instances and schemas are similar to types and variables in programming languages

- **Logical schema** – the overall logical structure of the database. Analogous to type information of a variable of a program.
- **Physical schema** – the overall physical structure of the database
- **Instance** – the actual content of the database at a particular point in time. Analogous to the value of a variable.

Somewhat comparable to classes and their instances.

Physical data independence

Physical data independence is the ability to modify the physical schema without changing the logical schema. Applications depend on the logical schema. However, interfaces between various levels and components should be well defined, so changes in some parts do not seriously influence others.

The separation of physical data and logical schema allows for increased flexibility and independence.

Database Languages

A database system provides a **data-definition language (DDL)** to specify the database schema and a **data-manipulation language (DML)** to express database queries and updates. In practice, the DDL and DML are not two separate languages, they just form parts of a single language (like SQL).

Data Definition Language

We can use a data definition language to describe our database schema through a set of definitions. For example

```
create table instructor(
        ID          char(5),
        name        varchar(20),
        dept_name   varchar(20),
        salary      numeric(8,2)
)
```

A DDL compiler generates a set of table templates stored in a **data dictionary** – an object which contains metadata such as database schema, integrity constraints, and authorization information.

Data Manipulation Language

A language for accessing and updating the data organized by the appropriate data model. Also known as a query language. There are two main types of data manipulation languages

- **Procedural DML** – which requires users to specify what data are needed and how to acquire them
- **Declarative / non-procedural DML** – which requires users to specify what data are needed without specifying how to acquire them (generally easier to learn and use than procedural)

The portion of a DML that involves information retrieval is known as a **query**.

The SQL Language

SQL is a non-procedural / declarative query language. A **query** takes as input a table (or tables) and always returns a single table. For example, suppose we wanted to find all instructors in the Computer Science department of a database

```
select name
from instructor
where dept_name = 'Comp. Sci'
```

Note that SQL is *not* a Turing machine equivalent language. The ability to compute complex functions in SQL is usually embedded in some higher-level language. Application programs usually access databases through language extensions or APIs.

Database Access from Application Programs

SQL, although useful, is not a Turing machine equivalent language. It doesn't support things like input from users, output to displays or communication over the network. Actions like these must be written in a **host language** — for example, C/C++/Java/Python — with embedded SQL queries that access the data in the database.

Application programs are programs that are used to interact with the database. They provide an abstraction that they can interact directly with the data stored in the database. Behind the scenes, the database will communicate with the OS, which in turn fetches data from storage.

Database Design

Database design is the process of designing the general structure of the database.

A high-level data model gives the database designer a conceptual framework that can be used to specify requirements of users. The initial phase of the design involves fully characterizing the needs of prospective users, then the designer chooses a model and creates a conceptual design.

The process of moving from an abstract data model to the implementation of the database proceeds in the **logical design phase** – in which the designer maps the high-level conceptual schema onto the implementation data model of the database system that will be used, and the **physical design phase**, in which the physical features of the database are specified.

Database Engine

A database system is partitioned into multiple modules which each have specific responsibilities. The functional components of a database system can be divided into the:

storage manager, query processor and transaction management components.

The storage manager is important because databases typically require a large amount of storage space. The query processor helps the database system to simplify and facilitate access to data, and allows database users to obtain good performance while being able to work at the view level. The transaction manager is important because it allows application developers to treat a sequence of database accesses as if they were a single unit that either happens entirely or not at all.

The Storage Manager

Provides the interface between low-level data stored in the database and the application programs and queries submitted. The storage manager is responsible for interaction with the OS file manager and efficient storage, retrieval and updating of data. Its subcomponents include the

- **Authorization and integrity manager** – tests for satisfaction of integrity constraints and checks for authority of users to access data
- **Transaction manager** – ensures that the database remains in a consistent (correct) state despite failures, and that concurrent transactions execute without conflicts
- **File manager** – manages allocation of space on disk storage and data structures used to represent information stored on disk
- **Buffer manager** – responsible for fetching data from disk storage into main memory, and deciding what to cache in main memory. Enables the database to handle data sizes that are much larger than the size of main memory, which is very critical.

The storage manager also implements various data structures as part of the physical system implementation:

- **Data files** – which store the database itself
- **Data dictionary** – which stores metadata about the structure of the database (schema)
- **Indices** – provide fast access to data items through pointers.

The Query Processor

The query processor components include

- **DDL interpreter** – interprets DDL statements and records their definitions in the data dictionary
- **DML compiler** – translates DML statements in a query language into an evaluation plan (low level instructions in a format that the query evaluation engine understands). It also performs query optimization (picks the lowest cost evaluation plan from all options available)
- **Query evaluation engine** – executes low level instructions generated by the DML compiler

Transaction Management

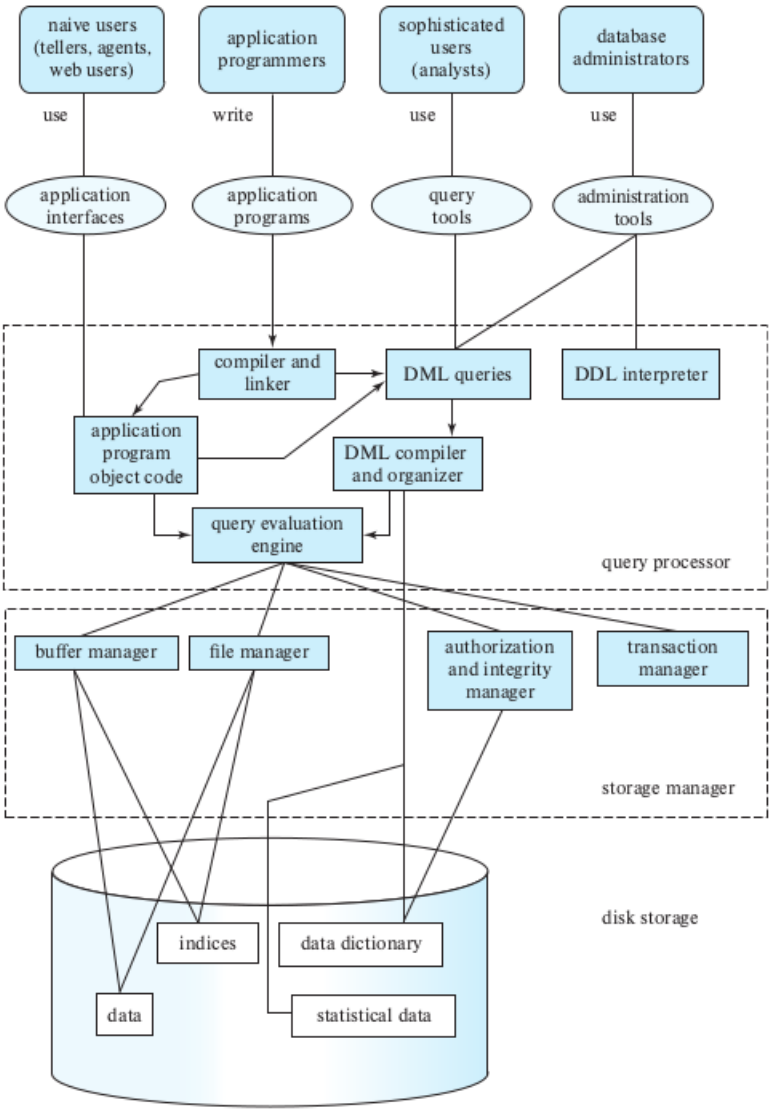
A **transaction** is a collection of operations that performs a single logical function in a database application. The transaction management component ensures that the database remains in a consist/correct states despite system failures and transaction failures. There is also a concurrency-control manager which controls the interaction among concurrent transactions (in order to minimize failures).

Database Architecture

There are multiple database architectures available.

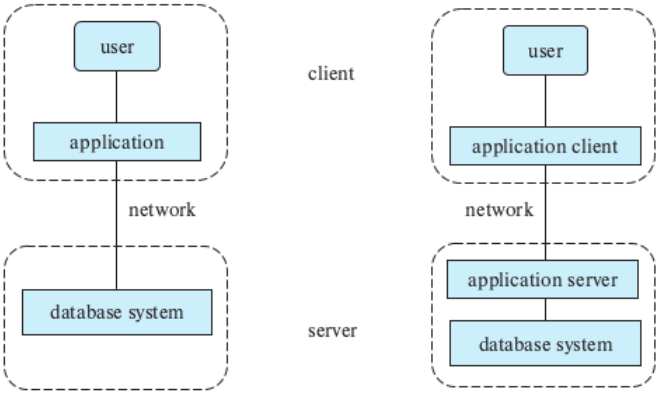
- **Centralized databases** – one to a few cores with shared memory
- **Client-server** – one server machine executes work on behalf on multiple client machines
- **Parallel databases** – multiple cores, shared memory, shared disk
- **Distributed databases** – geographically distributed databases with schema/data heterogeneity

The architecture of a database system that runs on a centralized server machine.



In terms of the architecture of applications that use databases as their backend. Database applications can be partitioned into two or three parts. Earlier generation database applications

used a **two-tier architecture**, where the application resides at the client machine, and invokes database system functionality at the server machine through query language statements. Modern database applications use a **three-tier architecture**, where the client machine acts only as a front end and doesn't contain any direct database calls. Web browsers and mobile apps are the more commonly used application clients today.



(a) Two-tier architecture (b) Three-tier architecture

Database Administrator

A **database administrator (DBA)** is a person who has central control over the database system. They have to perform the following jobs

- Schema definition
- Storage structure and access-method definition
- Schema and physical-organization modification
- Granting of authorization for data access
- Routine maintenance
- Periodically backing up the database
- Ensuring that enough free disk space is available for normal operations + upgrading disk space as required
- Monitoring jobs running on the database

2 | The Relational Model

The relational model remains the primary data model for commercial data processing applications. This is most likely due to its simplicity, which eases the job of the programmer.

Structure of Relational Databases

A relational database consists of a collection of **tables**, each of which has a unique name. Consider the *instructor* table, which stores information about instructors.

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci	92000
98345	Kim	Elec. Eng.	80000

The *instructor* table (relation) consists of **attributes** (columns), of which the *instructor* relation has four, and **tuples** (rows), where each row is essentially a series of information in a tuple. We use the term **relation instance** to refer to a specific instance of a relation.

For each attribute of a relation, the set of permitted values is called the **domain**. For example in the *instructor* relation, the *salary* attribute has the domain of all possible salary values, while the domain of the *name* attribute is the set of all possible instructor names. Attribute values are usually required to the **atomic** (that is, indivisible). The special value **null** is a member of every domain, which indicates that the value is unknown.

Database Schema

When talking about a database, we must differentiate between the **database schema** (the logical design) and the **database instance** (a snapshot of the data in the database at a given instance in time).

The concept of a relation corresponds to the programming-language notion of a variable, while the concept of a **relation schema** corresponds to the programming language notion of a type definition.

Keys

To express how tuples within a given relation are distinguished, we can use their attributes, or rather, attribute values of a tuple that can uniquely identify the tuple. It should not be the case that two tuples in a relation have exactly the same value for all attributes.

A **superkey** is a set of one or more attributes that, taken collectively, allow us to identify (uniquely) a tuple in the relation. For example, the *ID* attribute of the *instructor* relation is enough to distinguish one instructor tuple from another. Therefore *ID* is a superkey. The *name* attribute, on the other hand isn't, because two instructors might have the same name. However, the combination of both is also a superkey.

Formally, let R denote the set of attributes in the schema of relation r. If we say that a subset K of R is a *superkey* for r, if values for K are sufficient to identify a unique tuple of each possible relation $r(R)$.

Superkey K is a **candidate key** if K is *minimal* (that is, a super key with the minimum number of attributes). In the case of *instructor*, it's *ID*.

One of the candidate keys is chosen to be the **primary key (PK)** for the relation. The chosen key should be something static (phone numbers, emails, passport numbers can change over time and should not be used, despite their apparent stability). The primary key should also be something that is not modifiable by the user. It is customary to list the primary key attributes of a relation schema before other attributes. They are also underlined. Consider the schema for *instructor*

instructor (ID, name, dept_namem, salary)

Another type of constraint is called a **foreign-key constraint**. A foreign-key constraint from attribute(s) A of relation r_1 to the primary key B of relation r_2 states that on any database instance, the value of A for each tuple in r_1 must also be the value of $B$for some tuple in r_2. The attribute set A is called a **foreign-key** from r_1, referencing r_2. The foreign key in one relation must be the primary key at the referenced relation. Take for example, the *department* relation, in which *dept_name* acts as a foreign key for the *instructor* table

dept_name	*building*	*budget*
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

In this instance, the *instructor* relation is the **referencing relation**, as it is using the *dept_name* attribute from the *department* relation, which is called the **referenced relation**. In general, a **referential integrity constraint** requires that for the foreign key of your relation, when you go to the relation in which the foreign key is the primary key and query a row using that key, there must be at least one row/tuple returned.

Relational Algebra

Relational algebra is a procedural language consisting of a set of operations that take one or two relations as input and produce a new relation as their result. It has six basic operations.

The Select Operation

The **select** operation selects tuples that satisfy a given predicate. The Greek letter sigma (σ) is used to denote selection, and the predicate appears as a subscript to it. The argument relation is in parentheses after the σ.

Suppose we wanted to select all tuples of the *instructor* relation where the instructor is in the Physics department

$$\sigma_{dept_name = \text{"Physics"}}(instructor)$$

This would give us the following result:

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

We also allow comparisons using

$$=, \neq, >, \geq, <, \leq$$

in the selection predicate. We can combine several predicates into a larger one by using the connectives

$$\wedge \text{ (and)}, \vee \text{ (or)}, \neg \text{ (not)}$$

For example, if one wanted to find the instructors in Physics with a salary greater than $90,000, we would write

$$\sigma_{dept_name=\text{"Physics"} \wedge salary > 90000}(instructor)$$

The predicate can even include comparisons between two attributes.

The Project Operation

A unary operation that returns its argument relation, with certain attributes left out. Projection is denoted by the uppercase Greek letter pi (Π). We list those attributes that we wish to appear in the result as subscript to Π:

$$\Pi_{A_1, A_2, A_3, \ldots, A_k}(r)$$

where $A_1, A_2, \ldots, A_k$ are attribute names and r is a relation name. The result is defined as the relation of k columns obtained by erasing the columns that are not listed. Note that duplicate rows are removed from the result, as relations are sets.

Suppose we wanted to list all instructors' *ID*, *name* and *salary*, but we don't care about department name

$$\Pi_{ID, name, salary}(instructor)$$

Which would give us the following result

ID	*name*	*salary*
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

Expressions involving attributes can also appear as well. Suppose we wanted to get the monthly salary of each instructor

$$\Pi_{ID,name,salary/12}(instructor)$$

Composition of Relational Operators

The result of a relational-algebra operation is a relation, and therefore, operations can be composed together into **relational-algebra expressions**. Consider the query to find the names of all instructors in the Physics department

$$\Pi_{name}(\sigma_{dept_name=\text{"Physics"}}(instructor))$$

Instead of giving the name of a relation as the argument of the projection operation, we instead give an expression that evaluates to a relation.

The Cartesian Product Operation

The **Cartesian-product** operation, denoted by a cross ($\times$), allows us to combine information from any two relations. We write the cartesian product of relations r_1 and r_2 as $r_1 \times r_2$.

Take for example the result of the Cartesian product of *instructor* $\times$ *teaches*.

instructor.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2018
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
15151	Mozart	Music	40000	10101	CS-101	1	Fall	2017
15151	Mozart	Music	40000	10101	CS-315	1	Spring	2018
15151	Mozart	Music	40000	10101	CS-347	1	Fall	2017
15151	Mozart	Music	40000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
15151	Mozart	Music	40000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2017
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2018
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2017
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2018
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

Note that instead of producing pairs of tuples, the relational algebra concatenates data into a single tuple.

Since the name of attributes may be the same in both tables, we distinguish duplicates by attaching the name of the relation from which the attribute originally came from.

The Join Operation

Suppose we wanted to find the information about all instructors together with the *course_id* of all the courses they have taught. We would require the information in both the *instructor* relation and the *teaches* relation. The Cartesian product can do this, but it associates every instructor with ever course that was taught, regardless of whether or not the instructor taught that course.

However, we know that if an instructor has taught a course, then there is some tuple in

instructor × *teaches* that contains their name and sastisfies *instructor.ID* = *teaches.ID*. So writing

$$\sigma_{instructor.ID\ =\ teaches.ID}(instructor \times teaches)$$

We get the information we wanted (although with duplicate IDs, but that can be handled by the project operation).

instructor.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
32343	El Said	History	60000	32343	HIS-351	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-101	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-319	1	Spring	2018
76766	Crick	Biology	72000	76766	BIO-101	1	Summer	2017
76766	Crick	Biology	72000	76766	BIO-301	1	Summer	2018
83821	Brandt	Comp. Sci.	92000	83821	CS-190	1	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-190	2	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-319	2	Spring	2018
98345	Kim	Elec. Eng.	80000	98345	EE-181	1	Spring	2017

The **join** operation allows us to combine a selection and Cartesian product into a single operation. Given relations $r(R)$ and $s(S)$, let θ be a predicate on attributes in the schema $R \cup S$. The join operation $r \bowtie_\theta s$ is defined as follows

$$r \bowtie_\theta s = \sigma_\theta(r \times s)$$

Or for the one we just did

$$instructor \bowtie_{instructor.id=teaches.idteaches}$$

Set Operations

Set Union Operation

The union operation allows us to combine two relations. If we had two relations r and s, the notation would be

$$r \cup s$$

For this union to be valid

1. r and s must have the same *arity* (number of attributes)
2. The domains of r and s must be compatible. That is to say, for example, the second column of r has the same type of values as the second column of s.

As an example, if we wanted to find all courses taught in the Fall 2017 semester, or in the Spring 2018 semester, or in both

$$\Pi_{course_id}(\sigma_{semester=\text{"Fall"} \wedge year=2017}(section)) \cup \Pi_{course_id}(\sigma_{semester=\text{"Spring"} \wedge year=2018}(section))$$

Set Intersection Operation

The set-intersection operation allows us to find tuples that are in both the input relations. If we had two relations r and s, the notation would be

$$r \cap s$$

Again, for this intersection to be possible, the same conditions for the set union operation must be met (relations must be compatible).

Suppose we wanted to find the set of all courses taught in both the Fall 2017 semester and the Spring 2018 semesters. Using set intersection, we can write

$$\Pi_{course_id}(\sigma_{semester=\text{"Fall"} \wedge year=2017}(section)) \cap \Pi_{course_id}(\sigma_{semester=\text{"Spring"} \wedge year=2018}(section))$$

Set Difference Operation

The set-difference operation allows us to find tuples that are in one relation but not in another. If we had two relations r and s, the notation would be

$$r - s$$

For the difference to be possible, the relations must again be compatible. To find all courses taught in the Fall 2017 semester but *not* in the Spring 2018 semester

$$\Pi_{course_id}(\sigma_{semester=\text{"Fall"} \wedge year=2017}(section)) \cap \Pi_{course_id}(\sigma_{semester=\text{"Spring"} \wedge year=2018}(section))$$

The Assignment Operation

It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables. The can be done using the assignment operation — denoted by $\leftarrow$. It works similarly to variable assignment in programming. If we would like to find all instructors in the Physics and Music department

$$\begin{aligned}
&Physics \leftarrow \sigma_{dept_name=\text{"Physics"}}(instructor)\\
&Music \leftarrow \sigma_{dept_name=\text{"Music"}}(instructor)\\
&Physics \cup Music
\end{aligned}$$

Using this operation, a query can be written as a sequential program consisting of a series of assignments followed by an expression.

The Rename Operation

The results of relational-algebra expressions do not have a name that we can use to refer to them. We can give them names using the rename operator ρ. The expression

$$\rho_x(E)$$

returns the result of expression E under the name x. Another form of the rename operation

$$\rho_{x(A_1,A_2,\ldots,A_n)}(E)$$

returns the result of the expression E under the name x with the attributes renamed to

$$A_1, A_2, \ldots, A_n$$

This form can be used to give names to attributes in the results of relational algebra operations that involve expressions on attributes.

Equivalent Queries

There is more than a single way to write a query in relational algebra. For example, if we wanted to find information about courses taught by instructors in the Physics department with salaries greater than $90,000, it could be written as

$$\sigma_{dept_name=\text{"Physics"} \wedge salary > 90000}(instructor)$$

Or also as

$$\sigma_{dept_name=\text{"Physics"}}\left(\sigma_{salary > 90000}(instructor)\right)$$

Although the two queries are not identical, they are equivalent – they give the same result on any database.

3 | SQL

We will now study the most widely used database query language: SQL. Although it is referred to as a "query language", it can do much more – such as defining the structure of the data, modifying data in the database, and specifying security constraints.

Overview of SQL

The SQL language has several parts

- **Data-definition language (DDL)** – the SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schemas
- **Data-manipulation language (DML)** – provides the ability to query information from the database, and the ability to insert tuples into, delete tuples from and modify tuples in the database
- **Integrity** – the DDL includes commands for specifying integrity constraints (e.g. the use and enforcement of primary keys, as well as the uniqueness of primary keys)
- **View definition** – the DDL includes commands for defining views
- **Transaction control** – includes commands for specifying the beginning and ending of transactions (groups of instructions that should be reset if one of them fails, all or nothing)
- **Embedded SQL and dynamic SQL** – defines how SQL statements can be embedded within general-purpose programming languages
- **Authorization** – includes commands for specifying access rights to relations and views

SQL Data Definition

The SQL data-definition language (DDL) allows the specification of relations, and information about relations, such as

- The schema for each relation
- The type of values associated with each attribute
- The integrity constraints
- The set of indices to be maintained for each relation
- Security and authorization information for each relation
- The physical storage structure of each relation on disk

Basic Types

SQL has the following basic domain types

- `char(n)` – fixed length character string with user specified length n.
- `varchar(n)` – variable length character strings with user-specified maximum length n
- `int` – integer (a finite subset of the integers that is machine-dependent)
- `smallint` – a small integer (machine-dependent subset of the integer domain type)
- `numeric(p,d)` – fixed point number with user specified precision of p digits with d digits to the right of the decimal point
- `real`, `double` – floating point and double-precision floating point numbers, with machine-dependent precision
- `float(n)` – floating point number with user-specified precision of at least n digits

Basic Schema Definition

We define an SQL relation by using the `create table` command. The following command creates a relation *department* in the database

```
create table department
    (dept_name  varchar(20),
     building    varchar(15),
     budget      numeric(12,2),
     primary key (dept_name));
```

Basic Query Structure

A typical SQL query has the form

$$\begin{aligned}&\textbf{select } A_1, A_2, \ldots, A_n\\&\textbf{from } r_1, r_2, \ldots, r_m\\&\textbf{where } P\end{aligned}$$

where A_i represents an attribute, R_i represents a relation, P is a predicate. The result of this SQL query is a relation. The query above is essentially a project (Π) operation. The **select** clause lists the attributes desired in the result of a query. Corresponds to the projection operation of the relational algebra. If we wanted to find the name of all instructors

```
SELECT name from instructor;
```

SQL allows for duplicates in relations as well as in query results. To force the elimination of duplicates, we can insert the keyword **distinct** after our selection. To find the department names of all instructors, and remove duplicates

```
SELECT DISTINCT dept_name FROM instructor;
```

The keyword **all** specifies that duplicates should not be removed (or just omit the distinct keyword)

```
SELECT ALL dept_name FROM instructor;
```

We can select all attributes using the wildcard operator

```
SELECT * FROM instructor;
```

An attribute can be a literal with no `from` clause – the query below results in a table with one column and a single row with value `"437"`

```
SELECT '437';
```

We can give the column a name using

```
SELECT '437' as foo;
```

An attribute can be a literal with a `from` clause – the query below results in a table with one column and N rows (where N is the number of tuples in the *instructor* table), each row with value `"A"`

```
SELECT 'A' from instructor;
```

We can also select multiple attributes

```
SELECT id, name, salary FROM instructor;
```

We can apply expressions on attributes as well

```
SELECT id, name, salary/12 FROM instructor;
```

However, this gives us an ugly column name. We can fix this with an alias

```
SELECT id, name, salary/12 as monthly_salary FROM instructor;
```

With the `where` clause, we can get rows that satisfy a specific predicate

```
SELECT name FROM instructor WHERE dept_name='Comp. Sci.';
```

We can also use logical/comparison operators

```
SELECT name FROM instructor
WHERE dept_name='Comp. Sci.' AND salary > 70000;
```

We can also try to select from multiple tables as well – the result below is essentially the Cartesian product operation

```
SELECT * FROM instructor, teaches
```

Note that duplicate attributes will now have the name of the relation preceding it.

SQL allows renaming relations and attributes using the `as` clause (however, it can actually be omitted and the same result will be obtained)

```
old_name AS new_name
```

For example, to find the names of all instructors who have a higher salary than some instructor in Computer Science

```
SELECT DISTINCT T.name
FROM instructor AS T, instructor AS S
WHERE T.salary > S.salary and S.dept_name = "Comp. Sci.";
```

String Operations

SQL includes a string-matching operator for comparisons on character strings. The operator `LIKE` uses patterns that are described using two special characters – `%` matches any substring, `_` matches any character. Suppose we wanted to find all instructors whose name includes the substring `"dar"`

```
SELECT name
FROM instructor
WHERE name LIKE '%dar%';
```

However, string operations are often quite pricey, and should be avoided unless necessary.

Ordering Display of Tuples

When we make queries, we want to see data. At times, our view of the data can help a lot. Suppose we wanted to list in alphabetic order, the names of all instructors

```
SELECT DISTINCT name
FROM instructor
ORDER BY name;
```

We may specify `DESC` for descending order or `ASC` for ascending order for each attribute. By default, the order is ascending.

Where Clause Predicates

SQL includes a `BETWEEN` comparison operator as well. For example, if we wanted to find the names of all instructors with salary between $90,000 and 100{,}000$ (that is $90,000 \leq salary \leq 100,000$)

```
SELECT name
FROM instructor
WHERE salary BETWEEN 90000 AND 100000;
```

We can also use tuple comparison. For example, if we wanted to find instructors who are teaching biology.

```
SELECT name, course_id
FROM instructor, teaches
WHERE (instructor.ID, dept_name) = (teaches.ID, 'Biology');
```

Set Operations

To find courses that ran in Fall 2017 or in Spring 2018

```
(SELECT course_id
FROM section
WHERE semester = 'Fall' AND year = 2017)
INTERSECT
(SELECT course_id
FROM section
WHERE semester = 'Spring' AND year = 2018);
```

To find courses that ran in Fall 2017 and in Spring 2018

```
(SELECT course_id
FROM section
WHERE semester = 'Fall' AND year = 2017)
UNION
(SELECT course_id
FROM section
WHERE semester = 'Spring' AND year = 2018);
```

To find courses that ran in Fall 2017 but not in Spring 2018

```
(SELECT course_id
FROM section
WHERE semester = 'Fall' AND year = 2017)
EXCEPT
(SELECT course_id
FROM section
WHERE semester = 'Spring' AND year = 2018);
```

Null Values

It is possible for tuples to have a null value, denoted by `null`, for some of their attributes. `null` signifies an unknown value or that a value does not exist. The result of any arithmetic expression involving `null` is `null`

The predicate `is null` can be used to check for null values. For example, to find all instructors whose salary is `null`

```
SELECT name
FROM instructor
WHERE salary IS NULL;
```

The predicate `is not null` succeeds if the value on which it is applied is not null.

SQL treats the result of any comparison involving a null value as `unknown`. The result of `where` clause predicate is treated as false if it evaluates to `unknown`.

Aggregate Functions

These functions operate on the multiset of values of a column of a relation, and return a value: `avg, min, max, sum, count`.

To find the average salary of instructors in Computer Science

```
SELECT AVG(salary)
FROM instructor
WHERE dept_name = 'Comp. Sci.';
```

To find the total number of instructors who are teaching in Spring 2018

```
SELECT COUNT(DISTINCT ID)
FROM teaches
WHERE semester = 'Spring' AND year = 2018;
```

To find the number of tuples in the course relation (counting rows)

```
SELECT COUNT(*)
FROM course
```

Group By

We can also group data. Suppose we wanted to find the average salary of instructors in each department

```
SELECT dept_name, AVG(salary) as avg_salary
FROM instructor
GROUP BY dept_name;
```

It is generally used in conjunction with aggregate functions.

Having Clause

To find the names and salaries of all departments whose average salary is greater than $42000

```
SELECT dept_name, avg(salary) AS avg_salary
FROM instructor
GROUP BY dept_name
HAVING avg(salary) > 42000;
```

Predicates in the having clause are applied after the formation of groups, whereas predicates in the where clause are applied before forming groups.

Nested Subqueries

SQL provides a mechanism for nesting queries. A **subquery** is a select-from-where expression that is nested within another query. The nesting can be done in the following SQL query

```
select A_1, A_2, A_3, ..., A_n
from r_1, r_2, r_3, ..., r_m
where P
```

where r_i can be replaced by any subquery, P can be replaced by an expression of the form $B < operation > (subquery)$, where B is an attribute and <operation> is to be defined later and A_i can be replaced by a subquery that generates a single value.

For example, suppose we wanted to find courses offered in Fall 2017 and in Spring 2018, we could use a subquery (in the parens)

```
SELECT DISTINCT course_id
FROM section
WHERE semester = 'Fall' AND YEAR = 2017 AND
        course_id IN (SELECT course_id
            FROM section
            WHERE semester = 'Spring' AND year = 2018);
```

To find courses offered in Fall 2017 but not in Spring 2018

```
SELECT DISTINCT course_id
FROM section
WHERE semester = 'Fall' AND YEAR = 2017 AND
        course_id NOT IN (SELECT course_id
            FROM section
            WHERE semester = 'Spring' AND year = 2018);
```

Suppose we wanted to find all instructors whose name is neither "Mozart" nor "Einstein"

```
SELECT DISTINCT name
FROM instructor
WHERE name NOT IN ('Mozart', 'Einstein');
```

To illustrate the usage of nested subqueries, wwe could also find the total number of distinct students who have taken the course sections taught by instructor with ID `10101`

```
SELECT COUNT(distinct ID)
FROM takes
WHERE (course_id, sec_id, semester, year) IN
        (SELECT course_id, sec_id, semester, year
        FROM teaches
        WHERE teaches.ID = 10101
        );
```

To find the names of instructors with salaries greater than that of some (at least one) instructor in the Biology department, we can use predicates or the `some` clause with a subquery

```
/* Regular way */
SELECT DISTINCT T.name
FROM instructor AS T, instructor AS S
WHERE T.salary > S.salary AND S.dept_name = "Biology";

/* Some clause way */
SELECT name
FROM instructor
WHERE salary > SOME(select salary
    FROM instructor
    WHERE dept_name = "Biology");
```

To find the names of all instructors whose salary is greater than the salary of **all** instructors in the Biology department, we use the `all` clause.

```
SELECT name
FROM instructor
WHERE salary > ALL (SELECT salary
    FROM instructor
    WHERE dept_name = 'Biology');
```

Testing For Empty Relations

The *exists* construct returns the value *true* if the argument subquery is nonempty. We could use this to specify the query "Find all courses taught in both the Fall 2017 semester and in the Spring 2018 semester", for example

```
SELECT course_id
FROM section AS S
WHERE semester = 'Fall' AND year = 2017 AND
        EXISTS (SELECT *
            FROM section AS T
            WHERE semester = 'Spring'
                AND year = 2018
                AND S.course_id = T.course_id);
```

In this case, the variable *S* is known as the correlation name. The inner query is a correlated subquery. The same thing applies for "not exists", except we use the `except` keyword. To find all students who have taken all courses offered in the Biology department (the first query lists all the courses offered in Biology and the second nested on lists all courses a particular student took)

```
SELECT DISTINCT S.ID, S.name
FROM student AS S
WHERE NOT EXISTS (
    (SELECT course_id
    FROM course
    WHERE dept_name = 'Biology')
    EXCEPT
    (SELECT T.course_id
    FROM takes AS T
    WHERE S.ID = T.ID)
);
```

Subqueries In the From Clause

SQL allows a subquery expression to be used in the `from` clause. If we wanted to find the average instructors' salaries of those departments where the average salary is greater than $42,000

```
SELECT T.dept_name, T.avg_salary
FROM (
    SELECT dept_name, AVG(salary) AS avg_salary
    FROM instructor
    GROUP BY dept_name) AS T
WHERE T.avg_salary > 42000;
```

Another way to write to write it is

```
SELECT dept_name, avg_salary
FROM (
    SELECT dept_name, AVG(salary)
    FROM instructor
    GROUP BY dept_name
    ) AS dept_avg(dept_name, avg_salary)
WHERE avg_salary > 42000;
```

With Clause

The **with** clause provides a way of defining a temporary relation whose definition is available only to the query in which the with clause occurs. To find all departments with the maximum budget

```
WITH max_budget(value) AS (SELECT MAX(budget) FROM department)
SELECT department.dept_name
FROM department, max_budget
WHERE department.budget = max_budget.value
```

Scalar Subquery

A scalar subquery is one which is used where a single value is expected. To list all departments along with the number of instructors in each department

```
SELECT dept_name,
        (SELECT COUNT(*)
         FROM instructor
         WHERE department.dept_name = instructor.dept_name)
         AS num_instructors
FROM department;
```

Deletion

To delete all instructors

```
DELETE FROM instructor;
```

To delete all instructors in the Finance department

```
DELETE FROM instructor
WHERE dept_name = 'Finance';
```

To delete all tuples in the *instructor* relation for instructors associated with a department located in the Watson building

```sql
DELETE FROM instructor
WHERE dept_name IN (
    SELECT dept_name
    FROM department
    WHERE building = 'Watson'
);
```

To delete all instructors whose salary is less than the average salary of instructors

```sql
DELETE FROM instructor
WHERE salary < (SELECT AVG(salary) FROM instructor);
```

Note that as we delete tuples from instructor, the average salary changes. We can solve this problem by first computing the average of the salary, finding all the tuples to delete, then deleting all tuples.

Insertion

To add a new tuple to *course*

```sql
INSERT INTO course
VALUES ('CS-347', 'Database Systems', 'Comp. Sci.', 4);
```

Or equivalently

```sql
INSERT INTO course (course_id, title, dept_name, credits)
VALUES ('CS-347', 'Database Systems', 'Comp. Sci.', 4);
```

To add a new tuple to *student* with `tot_creds` set to null

```sql
INSERT INTO student
VALUES ('3003', 'Green', 'Finance', null);
```

To make each student in the Music department who has earned more than 144 credits hours an instructor in the Music department with a salary of $18,000

```sql
INSERT INTO instructor
SELECT ID, name, dept_name, 18000
FROM student
WHERE dept_name = 'Music' AND total_cred > 144;
```

The *select from where* statement is evaluated fully before any of its results are inserted into the relation. Otherwise, queries like `insert into table1 select * from table1` would cause problems.

Updates

To give a 5% salary raise to all instructors

```sql
UPDATE instructor
SET salary = salary * 1.05;
```

To give a 5% salary raise to instructors who earn less than $70,000

```sql
UPDATE instructor
SET salary = salary * 1.05
WHERE salary < 70000;
```

To give a 5% salary raise to instructors whose salary is less than the average

```sql
UPDATE instructor
SET salary = salary * 1.05
WHERE salary < (SELECT AVG(salary) FROM instructor);
```

To increase the salaries of instructors whose salary is over $100,000 by 3% and all others by 5%, we can write two update statements (note that the order is important)

```sql
UPDATE instructor
SET salary = salary * 1.03
WHERE salary > 100000;

UPDATE instructor
SET salary = salary * 1.05
WHERE salary <= 100000;
```

However, this does have an issue where someone can be promoted twice. We can use the *case* statement to avoid this problem

```sql
UPDATE instructor
SET salary = CASE
    WHEN salary <= 100000 THEN salary * 1.05
    ELSE salary * 1.03
end;
```

Views

A *view* provides a mechanism to hide certain data from the view of certain users. Any relation that is not of the conceptual model but is made visible to a user as a "virtual relation" is known as a view. It's essentially a temporary table.

Suppose we wanted to create a view of instructors without their salary

```sql
CREATE VIEW faculty AS
    SELECT ID, name, dept_name FROM instructor;
```

To find all instructors in the Biology department using our new faculty view

```sql
SELECT name
FROM faculty
WHERE dept_name = 'Biology';
```

To create a view of department salary totals

```sql
CREATE VIEW departments_total_salary(dept_name, total_salary) AS
    SELECT dept_name, SUM(salary)
    FROM instructor
    GROUP BY dept_name;
```

We can define a view using another view

```sql
CREATE VIEW physics_fall_2017 AS
        SELECT course.course_id, sec_id, building, room_number
        FROM course, section
        WHERE course.course_id = section.course_id
                AND course.dept_name = 'Physics'
                AND section.semester = 'Fall'
                AND section.year = '2017';

CREATE VIEW physics_fall_2017_watson AS
        SELECT course_id, room_number
        FROM physics_fall_2017
        WHERE building = 'Watson';
```

To expand the `physics_fall_2017_watson` view, we can do

```sql
CREATE VIEW physics_fall_2017_watson_expansion AS
SELECT course_id, room_number
FROM (SELECT course.course_id, building, room_number
    FROM course, section
    WHERE course.course_id = section.course_id
        AND course.dept_name = 'Physics'
        AND section.semester = 'Fall'
        AND section.year = '2017') AS T
WHERE building = 'Watson';
```

We can also update views. To create a view of instructors without their salary

```sql
CREATE VIEW faculty AS
SELECT ID, name, dept_name
FROM instructor;
```

To create a view of department salary totals

```sql
CREATE VIEW departments_total_salary(dept_name, total_salary) AS
SELECT dept_name, SUM(salary)
FROM instructor
GROUP BY dept_name;
```

Suppose we wanted to add a new tuple to the *faculty* view we defined earlier. This insertion would actually be inserted into the real *instructor* table, except the `salary` attribute would be null.

```sql
INSERT INTO faculty
VALUES ('30765',  'Green', 'Music');
```

Joining Relations

Join operations take two relations and return as a result another relation. It is a Cartesian product which requires that tuples in the two relations match (under some condition). It also specifies the attributes that are present in the result of the join. There are three types.

Natural Join

Natural join matches tuples with the same values for all common attributes, and retains only one copy of each common column. If were to naturally join the *student* and *takes* relations, it would find the ID as the common attribute:

ID	*name*	*dept_name*	*tot_cred*	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	Zhang	Comp. Sci.	102	CS-101	1	Fall	2017	A
00128	Zhang	Comp. Sci.	102	CS-347	1	Fall	2017	A-
12345	Shankar	Comp. Sci.	32	CS-101	1	Fall	2017	C
12345	Shankar	Comp. Sci.	32	CS-190	2	Spring	2017	A
12345	Shankar	Comp. Sci.	32	CS-315	1	Spring	2018	A
12345	Shankar	Comp. Sci.	32	CS-347	1	Fall	2017	A
19991	Brandt	History	80	HIS-351	1	Spring	2018	B
23121	Chavez	Finance	110	FIN-201	1	Spring	2018	C+
44553	Peltier	Physics	56	PHY-101	1	Fall	2017	B-
45678	Levy	Physics	46	CS-101	1	Fall	2017	F
45678	Levy	Physics	46	CS-101	1	Spring	2018	B+
45678	Levy	Physics	46	CS-319	1	Spring	2018	B
54321	Williams	Comp. Sci.	54	CS-101	1	Fall	2017	A-
54321	Williams	Comp. Sci.	54	CS-190	2	Spring	2017	B+
55739	Sanchez	Music	38	MU-199	1	Spring	2018	A-
76543	Brown	Comp. Sci.	58	CS-101	1	Fall	2017	A
76543	Brown	Comp. Sci.	58	CS-319	2	Spring	2018	A
76653	Aoi	Elec. Eng.	60	EE-181	1	Spring	2017	C
98765	Bourikas	Elec. Eng.	98	CS-101	1	Fall	2017	C-
98765	Bourikas	Elec. Eng.	98	CS-315	1	Spring	2018	B
98988	Tanaka	Biology	120	BIO-101	1	Summer	2017	A
98988	Tanaka	Biology	120	BIO-301	1	Summer	2018	*null*

Normally we would write this in SQL as

```
SELECT *
FROM student, takes
WHERE student.ID = takes.ID;
```

To same query in SQL with the natural join construct is

```
SELECT *
FROM student NATURAL JOIN takes;
```

However, we do need to be wary of unrelated attributes with the same name which get equated incorrectly. For example, if we wanted to list the names of students along with the titles of courses they have taken, you would do it like

```
SELECT name, title
FROM student NATURAL JOIN takes, course
WHERE takes.course_id = course.course_id
```

The following would be incorrect

```
SELECT name, title
FROM student NATURAL JOIN takes NATURAL JOIN course;
```

Inner/Outer Join

Outer join is an extension of the join operation that avoids loss of information. It computes the join and then adds tuples from one relation that do not match tuples in the other to the result. It also uses null values instead of just leaving out tuples that do not have matching attributes.

There are three forms of outer join

- **Left outer join** preserves tuples only in the relation named before (to the left) of the left outer join operation
- **Right outer join** preserves tuples only in the relation named after (to the right of the operation)

- **Full outer join** joins all tuples from both tables even if they don't have matching values

Join operations that match tuples with the same values on a specific attribute (and do not preserve nonmatched tuples) are called **inner-join** operations.

Transactions

A **transaction** consists of a sequence of query and/or update statements and is a 'unit' of work. The SQL standard specifies that a transaction begins implicitly when an SQL statement is executed.

The transaction must end with one of the following statements

- **Commit work** – the updates performed by the transaction become permanent in the database
- **Rollback work** – all the updates performed by the SQL statements in the transaction are undone

An **atomic transaction** is either fully executed or rolled back as if it never occurred. It has isolate from concurrent transactions.

Suppose we wanted to transfer money from one account to another. The process might look something like

1. Check to see if the first account has balance greater than the desired transfer amount
2. Subtract said transfer amount from the first account (database modified here)
3. Add the transfer amount to the second account
4. Commit changes, so that if something goes wrong beforehand, nothing happens

Integrity Constraints

Integrity constraints guard against accidental damage to the database by ensuring that authorized changes to the database do not result in a loss of data consistency. For example

- A checking account must have a balance greater than $10,000.
- The salary of a bank employee must be at least four dollars an hour.
- A customer must have a non-null phone number.

Constraints on a Single Relation

For a single relation, we have the following constraints

- `not null`
- `primary key` (must be unique and non null)
- `unique`
- `check (P)` where P is a predicate

Check

The `check (P)` clause specifies a predicate P that must be satisfied by every tuple in a relation. For example, if we wanted to ensure that semester is one of fall, winter, spring, or summer

```
SELECT name, title
FROM student NATURAL JOIN takes, course
WHERE takes.course_id = course.course_id
```

The following would be incorrect

```
CREATE TABLE sectionz
    (course_id VARCHAR(8),
     sec_id VARCHAR(8),
     semester VARCHAR(6),
     year NUMERIC(4,0),
     building VARCHAR(15),
     room_number VARCHAR(7),
     time_slot_id VARCHAR(4),
     PRIMARY KEY (course_id, sec_id, semester, year),
     CHECK(semester in ('Fall','Winter','Spring','Summer')));

/* Works */
INSERT INTO sectionz
        VALUES('iccs101', '1', 'Fall', '2023',  'Bldg1', '1302', '5');

/* Doesn't */
INSERT INTO sectionz
        VALUES('iccs202', '1', 'T1', '2023',  'Bldg1', '1502', '8');
```

Referential Integrity

Ensures that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation. For example, if "Biology" is a department name appearing in one of the tuples in the *instructor* relation, then there exists a tuple in the *department* relation for "Biology".

Let A be a set of attributes and R and S be two relations that contain attributes A and where A is the primary key of S. A is said to be a **foreign key** of R if for any values of A appearing in R, these values also appear in S. Foreign keys can be specified as part of the SQL `create table` statement.

When a referential-integrity constraint is violated, the normal procedure is to reject the action that caused the violation.

Authorization

We may assign a user several forms of authorization on parts of the database

- **Read**, **Insert**, **Delete**, **Update**

Each of these authorization types is called a **privilege**. Users can have all, none, or a combination of privileges.

4 | Database Design Using the E-R Model

The task of creating a database application is usually a complicated one, involving the design of a database schema, programs that access and update the data and security scheme to control access to data. The needs of the users play a central role in the design process. We will now focus on the design of the database schema.

Design Phases

A small relational database can probably be easily designed by a developer, but for more complicated systems, a high-level model can provide a conceptual framework which specifies the data requires of the database users and a database structure which satisfies those requirements. The design process generally includes the following phases

1. The first phase is to characterize the data needs of the prospective database users. The database designer must interact with potential users through interviews, document reviews, and revision of business processes. The outcome of this phase is a specification of user requirements.
2. Then the designer chooses a data model and translates the user requirements into a conceptual schema of the database. We will be using the entity-relational model to represent the conceptual design. The designer then reviews the schema to confirm all data requirements are met and do not conflict with each other, as well as removes any redundant features.
3. Next comes the journey from the conceptual design to the implementation. This involves two final design phases. In the *logical design phase*, the designer decides on a good database schema and maps the conceptual E-R model into said relation schema. Finally, the designer uses the resulting database schema in the *physical-design phases*, in which the physical layout of the database.

Design Alternatives

In designing a database schema, we must unsure that we avoid two major pitfalls

1. *Redundancy* – a bad design may result in repeated information. Redundant representation of information may lead to data inconsistency among the various copies of information.
2. *Incompleteness* – a bad design may make certain aspects of the enterprise difficult or impossible to model

Avoiding bad designs is not enough, there may be a large number of good designs which we can observe and use to follow best practices.

The Entity-Relationship Model

The **entity-relationship data model** was developed to facilitate database design by allowing specification of an **enterprise schema** which represents the overall structure of a database. The E-R data model uses three basic concepts: entity sets, relationship sets and attributes. It also has an associated diagrammatic representation.

Entity Sets

An **entity** is an object that exists and is distinguishable from other objects. For example, a specific person, company, event, plant, animal, etc. An **entity set** is a set of all entities of the same type which share the same properties. Each entity is represented by a set of attributes – descriptive properties possessed by all members of an entity set. For example

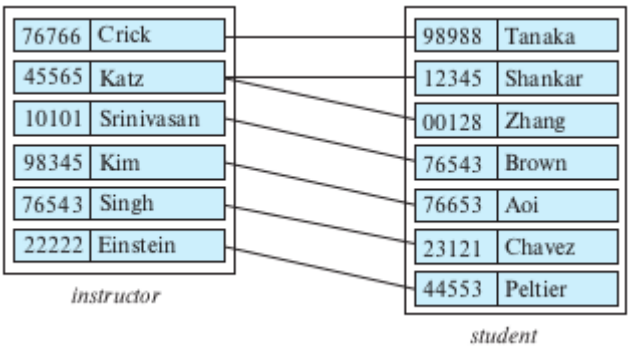
$$\textit{instructor} = (\text{ID, name, salary})$$
$$\textit{course} = (\text{course_id, title, credits})$$

Each entity has a **value** for each of its attributes. A subset of the attributes form a **primary key** of the entity set – a unique identifier to each member of the set, as well. A database thus includes a collection of entity sets.

An entity set is represented in an E-R diagram by a *rectangle*, which is divided into two parts. The first, shaded blue, contains the name of the entity set. The second part contains the names of all the attributes of the entity set.

Relationship Sets

A **relationship** is an association among several entities. A **relationship set** is a set of relationships of the same type. Suppose we wanted to consider two entity sets *instructor* and *student*. We could define the relationship set *advisor* to denote the associations between students and instructors who act as their advisors.



Note that, to keep the figure simple, only some of the attributes of the two entity sets are shown. A **relationship instance** in an E-R schema represents an association between the named entities in the real-world enterprise that is being model. For example, in the diagram above, the

instructor entity Katz with *ID* 45565, and the student entity Shankar, who has student *ID* 12345, participate in a relationship instance of *advisor*. This relationship instance represents that the instructor Katz is advising student Shankar.

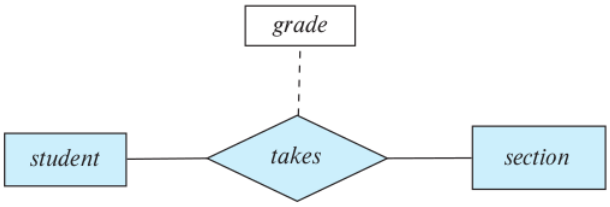
A relationship set is represented in an E-R diagram by a *diamond*, which is linked via *lines* to a number of different entity sets (rectangles).

Formally, a relationship set is a mathematical relation among $n \geq 2$ entities, each taken from entity sets. If $E_1, E_2, \ldots, E_n$ are entity sets, then a relationship R is a subset of

$$\{(e_1, e_2, \ldots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \ldots, e_n \in E_n\}$$

where $(e_1, e_2, \ldots, e_n)$ is a relationship instance. The association between entity sets is called **participation**, which means in this instance we say that the entity sets $E_1, E_2, \ldots, E_n$ **participate** in relationship set R.

A relationship may also include attributes known as **descriptive attributes**. As an example, consider the relationship set *takes*, which relates entity sets *student* and *section*. We may wish to store a descriptive attribute *grade*, with the relationship to record the grade that a student received in a course offering. An attribute of the relationship set is represented by an undivided rectangle, linked with a dashed line to the diamond representing the relationship set.



It is possible to have more than one relationship set involving the same entity sets. In this case the entity sets *student* or *section* might participate in a relationship set *teaching_assistant*, as well as the *takes* relationship set.

The relationship sets *advisor* and *takes* are examples of a **binary relationship set**, which are relationship sets involving two entity sets. The number of entity sets that participate in a relationship set is the **degree** of the relationship set.

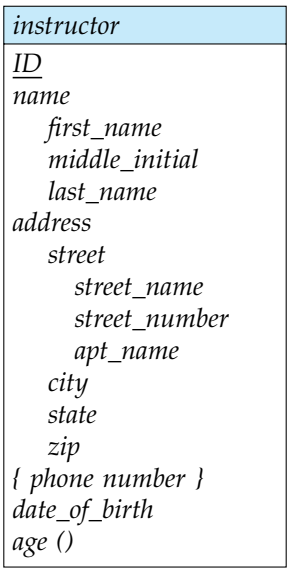
Complex Attributes

For every attribute, there is a set of permitted values called the **domain** or **value set** of that attribute. An attribute in an E-R model can be characterized by the following attribute types

- **Simple** and **composite** attributes. Simple attributes cannot be divided into subparts, unlike composite attributes, which can. Consider for example, the composite attribute *name*, which could consist of the attributes *first_name*, *middle_initial* and *last_name*.
- **Single** and **multi-valued** attributes. A multi-valued attribute may involve something like a phone number – of which a person may have multiple. On the other hand, a student ID is a single-valued attribute, as every student only has one.

- **Derived** attributes, which can be computed from other attributes. For example, age can be calculated from date of birth. Why not just store age? Age changes and date of birth doesn't.

Complex attributes can be represented in an E-R diagram as follows



An attribute takes on a **null** value when an entity doesn't have a value for it. The presence of a null value can indicate that said value does not exist for the entity, or that the value is missing. For example, if an instructor's name were to be null, we would assume it is missing, as every instructor must have a name. On the other hand, if an instructor's apartment number was null, we might assume they live in a house.

Mapping Cardinalities

Mapping cardinalities express the number of entities to which another entity can be associated via a relationship set. It is most useful in describing binary relationship sets. For a binary relationship set R between entity sets A and B, the cardinality mapping must be one of the following

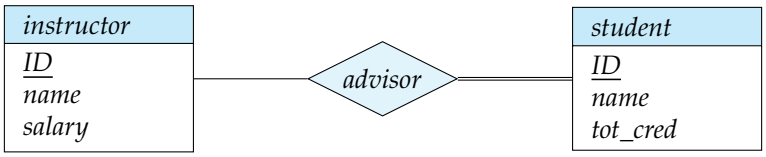
- **One-to-one** – an entity in A is associated with *at most* one entity in B, and an entity in B is associated with *at most* one entity in A
- **One-to-many** – an entity in A is associated with any number (≥ 0) of entities in B, however an entity in B can be associated with *at most* one entity in A.
- **Many-to-one** – an entity in A is associated with *at most* one entity in B, but an entity in B can be associated with ≥ 0 number of entities in A.
- **Many-to-many** – an entity in A is associated with ≥ 0 of entities in B, and vice versa.

In an E-R diagram, we express cardinality constraints by drawing either a direction line (→) signifying "one", or an undirected line (—) signifying "many", between the relationship set and the entity set. Take for example, a one-to-many relationship between an instructor and student

While a many to one relationship would be in the opposite direction

The participation of an entity set E in a relationship set R is said to be **total** if every entity in E must participate in at least one relationship in R. If this is not the case, the participation is said to be **partial**. For example, if a university requires every student to have an advisor, this means that the participation of *student* in the relationship set *advisor* is total. However, not all instructors need to have advisees, therefore we say that the participation of *instructor* in *advisor* is partial.

In an E-R diagram, total participation is shown using double lines.



It's also possible to indicate more complex constraints on the number of times each entity participates in relationships in a relationship set. A line may have an associated minimum and maximum cardinality, shown in the form $l..h$, where l and h are the minimum and maximum cardinality, respectively.

A maximum value of 1 indicates that the entity participates in at most one relationship, while a maximum value $*$ indicates no limit.

Primary Key

To specify how entities within a given entity set and relationships within a given relationship set are distinguished, we can make use of primary keys.

Primary Key for Entity Sets

By definition, individual entities are distinct. From a database perspective, the differences among them must be expressed in terms of their attributes. The values of the attribute values of an entity must be such that they can *uniquely identify* the entity. In other words, no two entities in an entity set are allowed to have exactly the same value for all attributes.

Therefore, the notion of a key, superkey, candidate key, and primary key are applicable to entity sets just as they are applicable to relation schemas. Keys also help to identify relationships uniquely, and thus distinguish relationships from each other.

Primary Key for Relationship Sets

To distinguish among the various relationships of a relationship set, we use the individual primary keys of the entities in the relationship set. Let R be a relationship set involving entity set $E_1, E_2, \ldots, E_n$. The primary key for R consists of the union of the primary keys of entity sets $E_1, E_2, \ldots, E_n$

$$\textit{primary-key}(E_1) \cup \textit{primary-key}(E_2) \cup \cdots \cup \textit{primary-key}(E_n)$$

If the relationship set R has attributes $a_1, a_2, \ldots, a_m$ associated with it, then the primary key of R also includes the attributes $a_1, a_2, \ldots, a_m$

$$\textit{primary-key}(E_1) \cup \textit{primary-key}(E_2) \cup \cdots \cup \textit{primary-key}(E_n) \cup \{a_1, a_2, \ldots, a_m\}$$

Choice of Primary Key for Binary Relationships

The choice of primary key for a binary relationship set depends on the mapping cardinality of the relationship set.

- For many-to-many relationships, the preceding union of the primary keys is a minimal superkey and is chosen as the primary key
- For one-to-many and many-to-one relationships, the primary key of the "many' side is a minimal superkey and is used as the primary key.
- For one-to-one relationships, the primary key of either one of the participating entity sets forms a minimal superkey, and either one can be chosen as the primary key to the relationship set.

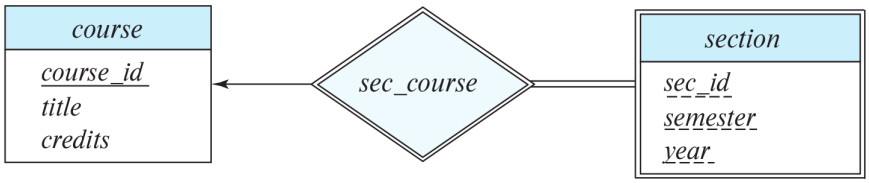
Weak Entity Sets

Consider a *section* entity, which is uniquely identified by a course identifier, semester, year and section identifier. Section entities are related to course entities, naturally. If we created a relationship set *sec_course* between entity sets *section* and *course*, we would notice that it is redundant, since *section* already has an attribute *course_id* that identifies the courses with which the section is related. We could get rid of *sec_course*, but then the relationship becomes implicit through the attribute, which isn't good.

Alternatively, we could take out *course_id* from the *section* entity. This would lead to the entity set *section* not having enough attributes to identify a particular *section* entity uniquely (since section ID, year and semester can be the same). To deal with this, we treat the relationship *sec_course* as a special relationship that provides extra information (in this case *course_id*, required to identify *section* entities uniquely.

This is what a **weak entity set** is, one whose existence is dependent on another entity set, called its **identifying entity set**. Instead of associating a primary key with a weak entity, we use the identifying entity, along with extra attributes called **discriminator attributes** to uniquely identify a weak entity. An entity set that is not a weak entity set is called a **strong entity set**.

In E-R diagrams, a weak entity set is depicted by a double rectangle. We underline the discriminator(s) of the weak entity set with a dashed line. The relationship set connecting the weak entity set to the identifying strong entity set is depicted by a double diamond.



The figure above also illustrates the usage of double lines to indicate that the participation of the (weak) entity set *section* in the relationship *sec_course* is total, meaning that every section must be related via *sec_course* to some course. Finally, the arrow from *sec_course* to course indicates that each section is related to a single course.

To generalize this, a weak entity set must have a total participation in its identifying relationship set, and the relationship must be many-to-one towards the identifying entity set.

5 | Relational Database Design

Well-Structured Relations

Well-structured relations contain minimal data redundancy and allow users to insert, delete and update rows without causing data inconsistencies. The goal is to avoid anomalies – weird things that can happen when the database is poorly designed. Some anomalies that can happen are

- **Insertion anomaly** – adding new rows forces the user to create duplicate data.
- **Deletion anomaly** – deleting rows may cause a loss of data that would be needed for other future rows.
- **Modification anomaly** – changing data in a row forces changes to other rows because of duplication.

For example, if we had the table

EmpID	Name	DeptName	Salary	CourseTitle	DateCompleted
100	Margaret Simpson	Marketing	48000	SPSS	6/19/2018
100	Margaret Simpson	Marketing	48000	Surveys	10/7/2018
140	Alan Beeton	Accounting	52000	Tax Acc	12/8/2018
110	Chris Lucero	Info Systems	43000	Visual Basic	1/12/2018
110	Chris Lucero	Info Systems	43000	C++	
190	Lorenzo Davis	Finance	55000		
150	Susan Martin	Marketing	42000	SPSS	6/19/2018
150	Susan Martin	Marketing	42000	Java	8/12/2018

If one were to delete Alan Beeton (suppose that he left), the course Tax Account would just *disappear*, assuming of course it was not stored elsewhere. If someone wanted to add a new course with an instructor, they'd also have to add the salary and department of the instructor as well, which is unnecessary. If we wanted to update the salary of an instructor – say Margaret Simpson – we'd have to do it in multiple places.

In conclusion, there are a plethora of issues, which something called normalization can fix.

Normalization

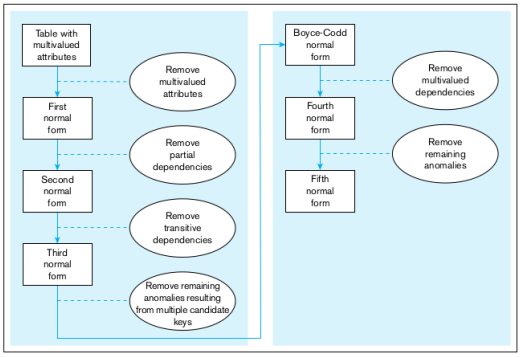
Normalization is a tool to validate and improve a logical design so that it satisfies certain constraints that avoid unnecessary duplication of data. Naturally, we cannot always avoid duplication of data – a foreign key, by definition, can be found in multiple places. However, we

would like to limit the duplication where it is not required or wasteful. This is done by reducing relations with anomalies to obtain smaller, well-structure relations.

Steps in Normalization

Normalization can be accomplished and understood in stages, each of which corresponds to a *normal form*. A normal form is a state of a relation that requires certain rules about relationships/dependencies between attributes are satisfied. There are

- **First normal form** – any multivalued attributes have been removed
- **Second normal form** – any partial functional dependencies have been removed (non-key attributes are identified by the whole primary key)
- **Third normal form** – any transitive dependencies have been removed (non-key attributes are identified by only the primary key)
- **Boyce-Codd normal form** – any remaining anomalies that result from functional dependencies are removed (as there was more than one possible primary key for the same non-keys)
- **Fourth normal form** – any multivalued dependencies have been removed
- **Fifth normal form** – any remaining anomalies have been removed



Generally, third-normal form is a good goal to achieve.

Functional Dependencies and Keys

A **functional dependency** is when the value of one attribute (the determinant) determines the value of another attribute. For any relation R, attribute B is functionally dependent A if for every valid instance of A, the value of A uniquely determines the value of B. This is represented by an arrow

$$A \rightarrow B$$

This does not mean that B can be computed from A, but rather if you know the value of A, there can be only one value for B. The attribute on the left of the arrow in a functional dependency is called the **determinant**. The attribute on the right is called the **dependent**.

Candidate Keys

Candidate keys are another unique identifier. They are an attribute, or combination of attributes, that uniquely identifies a row in a relation. A candidate key must satisfy the following properties

1. *Unique identification* – for every row, the value of the key must uniquely identify that row.
2. *Nonredundancy* – no attribute in the key can be deleted without destroying the property of unique identification.

Normalization Example

Let us explain normalization using an example of a customer invoice.

Represent the View in Tabular Form

The first step is to represent the user view as a single view or a relation, with the attributes recorded as column headings.

OrderID	Order Date	Customer ID	Customer Name	Customer Address	ProductID	Product Description	Product Finish	Product StandardPrice	Ordered Quantity
1006	10/24/2015	2	Value Furniture	Plano, TX	7	Dining Table	Natural Ash	800.00	2
					5	Writer's Desk	Cherry	325.00	2
					4	Entertainment Center	Natural Maple	650.00	1
1007	10/25/2015	6	Furniture Gallery	Boulder, CO	11	4–Dr Dresser	Oak	500.00	4
					4	Entertainment Center	Natural Maple	650.00	3

Convert to First Normal Form

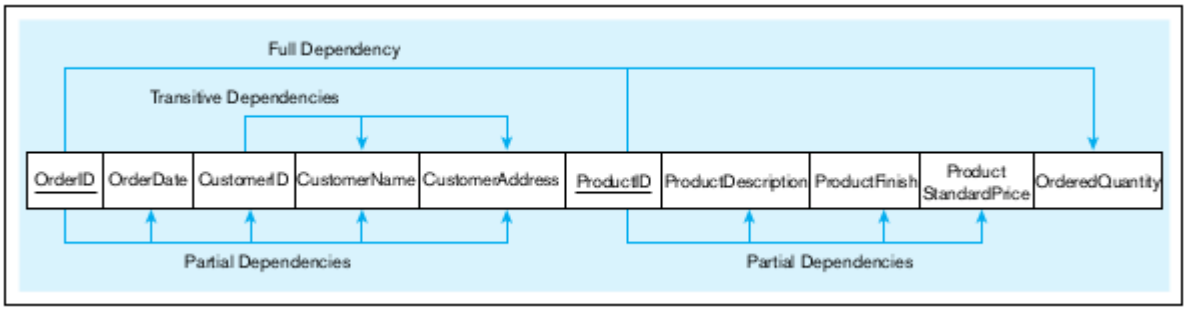
A relation is in **first normal form** if the following two constraints both apply:

- There are no repeating groups in the relation
- The primary key has been defined, which uniquely identifies each row in the relation
- Every value is atomic

To remove repeating groups, we fill relevant data into vacant cells in the table. To determine the primary key, we first observe that there are four determinants in the invoice, and their functional dependencies are

$$\text{OrderID} \rightarrow \text{OrderDate, CustomerID, CustomerName, CustomerAddress}$$
$$\text{CustomerID} \rightarrow \text{CustomerName, CustomerAddress}$$
$$\text{ProductID} \rightarrow \text{ProductDescription, ProductFinish, ProductStandardPrice}$$
$$\text{OrderID, ProductID} \rightarrow \text{OrderedQuantity}$$

Why do we know these are the functional dependencies? These business rules come from the organization.



We have the following anomalies in 1NF

- **Insertion anomaly** – with this setup, the company is unable to add new products and add it to the database before it is ordered for the first time, since no entries can be added to the table without both ProductID and OrderID
- **Deletion anomaly** – if a customer asks to delete the Dining Table from her OrderID 1006, this row must be deleted, and we will lose all information about the item's finish (Natural Ash) and price ($800).
- **Update anomaly** – if the company decides to increase the price of their Entertainment Center (ProductID 4), this change must be recorded in all rows containing that item.

Convert to Second Normal Form

A relation is in **second normal form** if it is in first normal form and contains no partial dependencies. A **partial functional dependency** exists when a nonkey attribute is functionally dependent on part (but not all) of the primary key. We can see that the following partial dependencies exist above

$$\text{OrderID} \rightarrow \text{OrderDate, CustomerID, CustomerName, CustomerAddress}$$
$$\text{ProductID} \rightarrow \text{ProductDescription, ProductFinish, ProductStandardPrice}$$

To convert to second normal form, we must

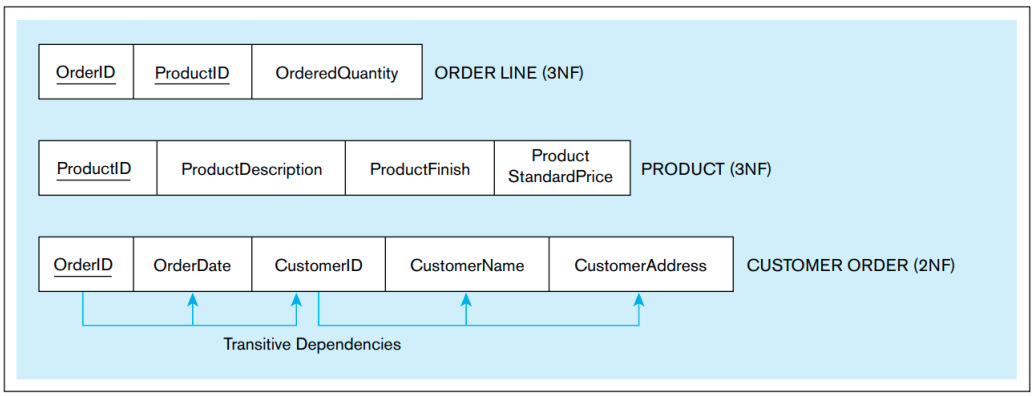
1. Make a new relation for each primary key attribute(s) that is a determinant in a partial dependency. That attribute becomes the primary key in the new relation.
2. Move the nonkey attributes that are only dependent on the primary key attribute(s) to the new relation from the old one.

Performing these steps for the INVOICE relation, we create two new relations – PRODUCT and CUSTOMER ORDER. The INVOICE is now left with the primary key attributes (OrderID and ProductID) and OrderedQuantity (which is functionally dependent on the whole key). We rename this relation ORDER LINE (since each row in the table represents one line item on an order). PRODUCT and ORDER LINE are now in third order form, but CUSTOMER ORDER is still not yet there.

Note that a relation that is in first normal form will be in second normal form if any one of the conditions applies

- The primary key consists of only one attribute (since there can't be a partial dependency in such a relation, by definition)

- No nonkey attributes exist in the relation (so all of the attributes in the relation are components of the primary key), which means that no functional dependencies exist
- Every nonkey attribute is functionally dependent on the full set of primary key attribtues (e.g. the attribute OrderedQuantity in the ORDER LINE relation)



Convert to Third Normal Form

A relation is in **third normal form** if it is in second normal form and no transitive dependencies exist. A **transitive dependency** is a functional dependency between the primary key and one or more nonkey attributes that are dependent on the primary key via another nonkey attribute. There are two in the CUSTOMER ORDER relation above

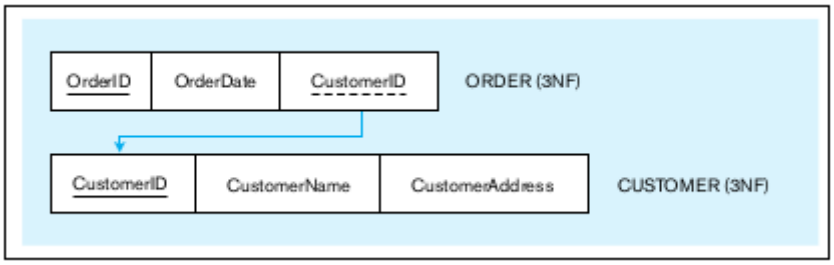
$$\text{OrderID} \rightarrow \text{CustomerID} \rightarrow \text{CustomerName}$$
$$\text{OrderID} \rightarrow \text{CustomerID} \rightarrow \text{CustomerAddress}$$

Transitive dependencies create unnecessary redundancy that may lead to the type of anomalies discussed earlier. For example, the transitive dependency in CUSTOMER ORDER requires that a customer's name and address be reentered every time a customer submits a new order, regardless of how many times they have been entered previously.

We can get rid of transitive dependencies through a three step procedure

1. For each nonkey attribute (or set of attributes) that is a determinant in a relation, make a new relation. The attribute(s) become the primary key(s) for the new relation
2. Move all of the attributes that are functionally dependent only on the primary key of the new relation from the old to the new relation
3. Leave the attribute that serves a a primary key in the new relation in the old relation to act as a foreign key that can be associated to the new relation

By doing so to our CUSTOMER ORDER, we create a new relation named CUSTOMER to receive the components of the transitive dependency. CustomerID becomes the primary key of the relation and the attributes CustomerName and Customer Address are moved to it. CUSTOMER ORDER is renamed ORDER for simplicity and the attribute CustomerID remains as a foreign key in that relation (so we can associate an order with the customer who submitted the order). We obtain the following:


OrderID
OrderDate
CustomerID
ORDER (3NF)
CustomerID
CustomerName
CustomerAddress
CUSTOMER (3NF)